McMaster University
Faculty of Engineering
COMPSCI 4ZP6 - Capstone Project

# Design, Validation & Verification Document

# ARDEN

AI-Driven Game Narrative Editor

---

**Team 30**

| | |
|---|---|
| **Cathy Xu** | xu518@mcmaster.ca |
| **Duzhi Wu** | wud72@mcmaster.ca |
| **Moein Roghani** | roghanim@mcmaster.ca |
| **Xingjian Zheng** | zhengx68@mcmaster.ca |

Version 0
January 22, 2025

Capstone Instructor: Dr. Mehdi Moradi

# Table of Contents

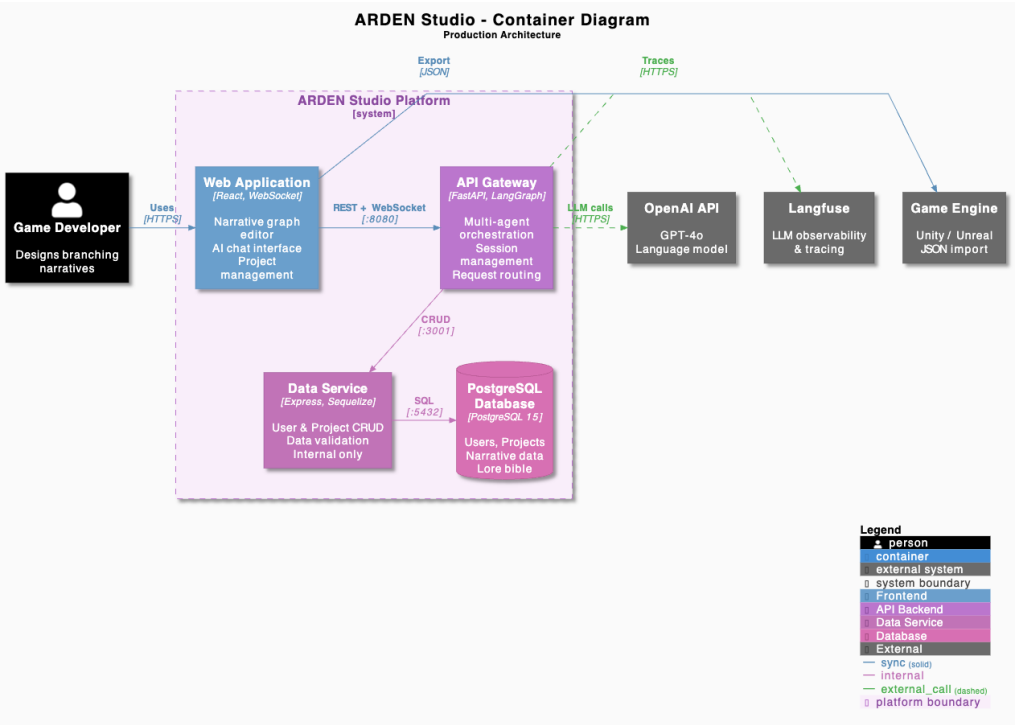
# 1 Design Specifications

## 1.1 System Architecture Overview

ARDEN Studio is an AI-powered narrative design tool that enables game developers to create branching storylines and manage world-building content through natural language interaction. This document details the system's design specifications and validation approach, following the **C4 model** for visualizing software architecture—a hierarchical approach with two levels of abstraction:

- **Containers**: Deployable units (applications, services, databases) and their interactions with external systems
- **Components**: Internal modules within each container

Figure 1.1 presents the container-level view of ARDEN Studio, showing how each deployable unit interacts within the platform boundary and with external systems.




| Container | Section | Component Diagram |
|---|---|---|
| Web Application | Frontend | Figure 1.3 |
| API Gateway | Backend API | Figure 1 |
| Data Service | Data Service | Figure 2 |

## 1.2 Backend API (arden-studio-api)

FastAPI application serving as the central gateway. Handles session management, AI agent orchestration, and proxies CRUD operations to the Data Service.

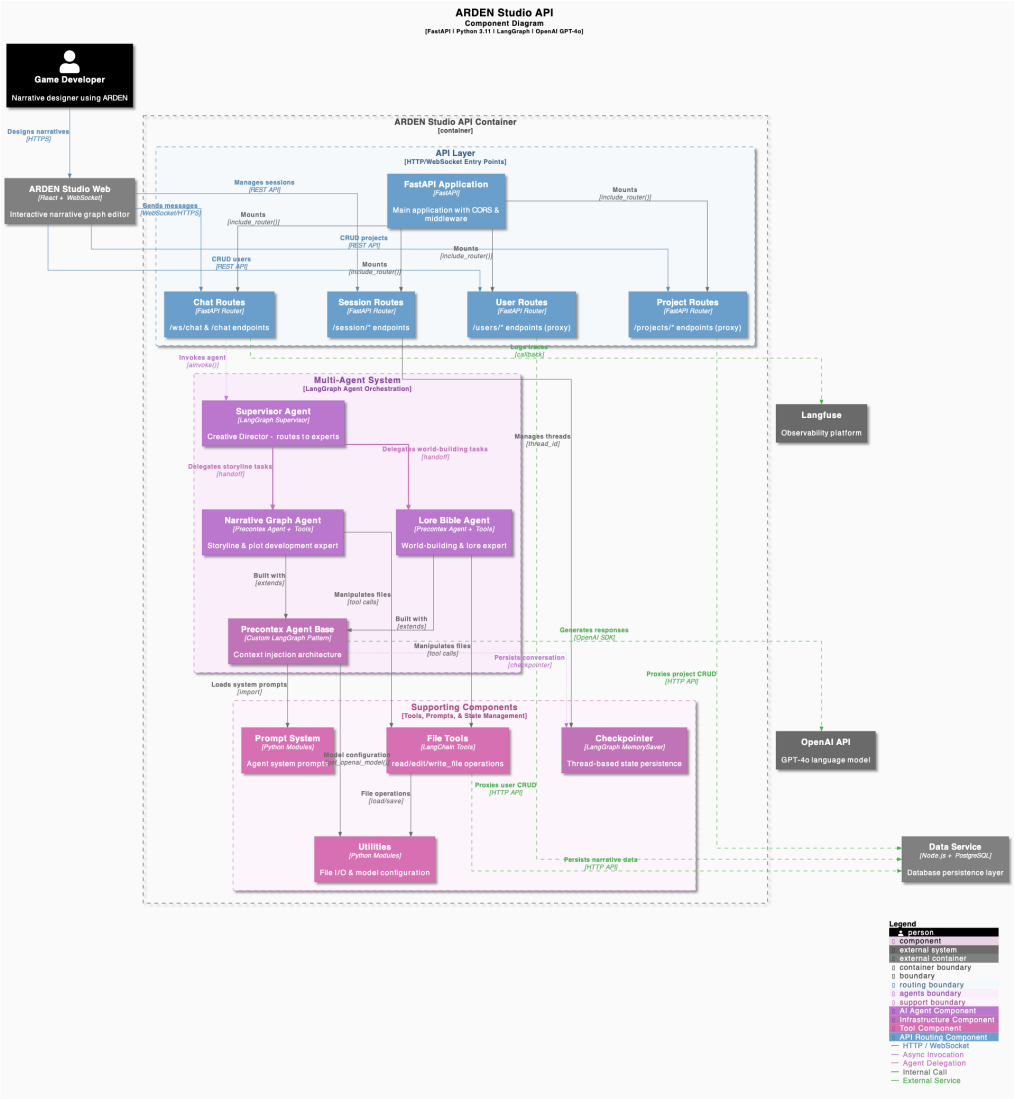



Figure 1: C4 Component Diagram: Backend API architecture.

**Technology Stack:** Python 3.11, FastAPI, LangGraph, LangChain, OpenAI GPT-4o

**Ports:** 4634 (development), 8080 (production)

### 1.2.1 API Endpoints

Base path: `/arden/api`

**Session Management**

| | |
|---|---|
| POST | /session/start |
| GET | /session/{id} |
| DELETE | /session/{id} |
| GET | /sessions |

**Users** *(proxy)*

| | |
|---|---|
| GET | /users |
| GET | /users/{id} |
| POST | /users |
| PUT | /users/{id} |
| DELETE | /users/{id} |

**Projects** *(proxy)*

| | |
|---|---|
| GET | /projects |
| GET | /projects/{id} |
| POST | /projects |
| PUT | /projects/{id} |
| DELETE | /projects/{id} |

**Chat**

| | |
|---|---|
| POST | /chat |
| WS | /ws/chat/{session_id} |

### 1.2.2 Implementation

**Core Classes**

| Class | Module | Responsibility |
|---|---|---|
| create_app() | app_factory | Factory function. Configures FastAPI, CORS, static files, mounts routers. |
| arden_supervisor | agents.agent | LangGraph Supervisor. Routes requests to expert agents via delegation. |
| precontex_agent_base() | bases | Factory function. Creates agent graph with context injection node. |
| BaseFileTool | tools.base | Abstract base. Provides validation, JSON checking, file I/O. |
| ReadFileTool | tools.read_file | Reads lore/storyline JSON. Returns content with line numbers. |
| EditFileTool | tools.edit_file | Atomic find-replace. Sequential edits, rollback on failure. |
| WriteFileTool | tools.write_file | Overwrites file. Validates JSON before write. |

| | | |
|---|---|---|
| `BaseAgentPrompts` | `prompts.base` | Abstract base for prompt classes. |
| `SupervisorPrompts` | `prompts` | System prompt for supervisor. Defines delegation rules. |
| `NarrativeGraphPrompts` | `prompts` | System prompt for narrative agent. DAG node/edge schema. |
| `LoreBiblePrompts` | `prompts` | System prompt for lore agent. World-building schema. |

### Multi-Agent System

| Agent | Model | Function |
|---|---|---|
| Supervisor | GPT-4o | Analyzes intent. Delegates to experts via `delegate_to_*` tool calls. Coordinates workflows. |
| Narrative Graph Expert | GPT-4o | Manages storyline DAG. Creates/edits nodes and edges. |
| Lore Bible Expert | GPT-4o | Manages canonical data. Creates/edits characters, locations, organizations, items, races. |

### Agent Graph Flow (precontex_agent_base)

1. `context_node`: Executes `context_function(message)` → returns `SystemMessage`
2. `agent`: LLM invocation with prompt + context + history
3. `tools`: If tool calls present, execute and loop back to agent
4. `END`: Return final response

### State Persistence

- `MemorySaver`: In-memory checkpointer. Persists conversation per `thread_id`.
- `sessions`: Dict-based session store. Maps `session_id` → `thread_id`.

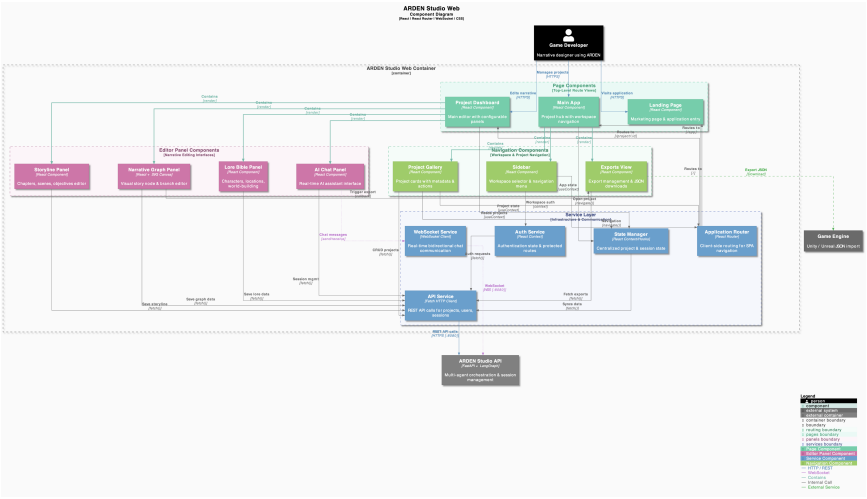
### External Dependencies

| Service | Protocol | Usage |
|---|---|---|

| | | |
|---|---|---|
| OpenAI API | HTTPS | LLM inference for all agents. Model: `gpt-4o-2024-05-13`. |
| Langfuse | HTTPS | Tracing, observability. Callback handler on chat routes. |
| Data Service | HTTP (internal) | Project/user CRUD. File tools persist narrative data. |

## 1.3 Frontend (arden-studio-web)

React single-page application providing the user interface for narrative design. Handles client-side routing, state management, and real-time communication with the API Gateway.



**Technology Stack:** React 18, React Router, WebSocket, CSS3

**Ports:** 3000 (development), 80 (production)

### 1.3.1 Client Routes

**Navigation Routes**

| | |
|---|---|
| `/` | Landing page |
| `/app` | Application shell |
| `/project/:id` | Project workspace |

**Internal View States**

| | |
|---|---|
| `home` | Project gallery grid |
| `downloads` | File export browser |

**Dashboard Panels**

| | |
|---|---|
| `graph` | Narrative node graph editor |
| `chat` | AI assistant interface |
| `lore` | World-building manager |
| `storyline` | Chapter/scene hierarchy |

### 1.3.2 Implementation

**Component Architecture**

| Component | File | Function |
|---|---|---|
| LandingPage | LandingPage.js | Entry page with animated background |
| MainApp | MainApp.js | Application shell; manages navigation state |
| Sidebar | Sidebar.js | Workspace selector; navigation menu |
| ProjectGallery | ProjectGallery.js | Project card grid |
| ProjectDashboard | ProjectDashboard.js | Project workspace; resizable split-panel layout |
| GraphPanel | GraphPanel.js | SVG-based narrative node graph |
| ChatbotPanel | ChatbotPanel.js | AI chat interface |
| LoreBiblePanel | LoreBiblePanel.js | Categorized world-building content |
| StorylinePanel | StorylinePanel.js | Hierarchical chapter/scene structure |
| DownloadsView | DownloadsView.js | File tree browser |

### 1.3.3 UI and User Experience

**Entry Flow** (Figure 8)

| View | Behavior | Known Issues |
|---|---|---|
| Landing Page | Animated background responds to cursor. CTA navigates to Project Hub. | Performance on low-end devices. |
| Project Hub | Project cards with title, genre, node count. Card selection enters workspace. | Mock data only. No pagination. |

**Navigation & Dashboard** (Figure 9)

| Component | Behavior | Known Issues |
|---|---|---|

| | | |
|---|---|---|
| Sidebar | Persistent navigation. Workspace selector, New Project, Home, Exports. | No keyboard navigation. |
| Dashboard | Dual-panel with draggable divider. Tab bar for panel configs. | State not persisted on refresh. |

**Graph & AI Panels** (Figure 10)

| Panel | Behavior | Known Issues |
|---|---|---|
| Narrative Graph | SVG nodes (Start, End, Dialogue, Choice, Action). Pan, zoom, fullscreen. | Node dragging not implemented. |
| AI Assistant | Chat interface. Suggestion chips. Enter submits. | Mock responses only. |

**Content Panels** (Figure 11)

| Panel | Behavior | Known Issues |
|---|---|---|
| Lore Bible | Categories: Characters, Locations, Races. Search, importance indicators. | CRUD non-functional. |
| Storyline Panel | Hierarchical chapters/scenes. Status, duration, content counts. | No drag-drop reordering. |

**Exports** (Figure 12)

| View | Behavior | Known Issues |
|---|---|---|
| Exports View | File tree: NarrativeProjects, Exports, Templates. JSON preview. | Download non-functional. |

## 1.4 Data Service (arden-data-service)

Node.js/Express REST API managing persistent storage via PostgreSQL. Exposes HTTP endpoints for CRUD operations on users and projects. Sequelize ORM enforces relational integrity.

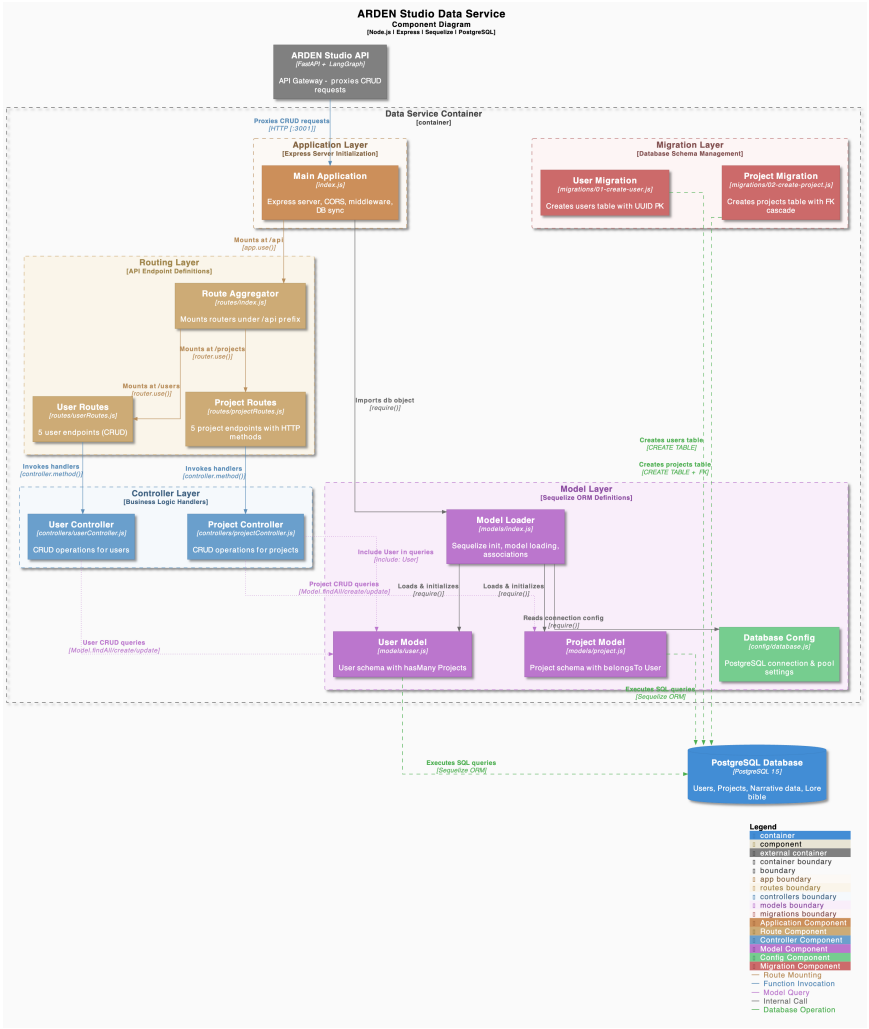



Figure 2: C4 Component Diagram: Data Service layer architecture.

**Technology Stack:** Node.js, Express, Sequelize ORM, PostgreSQL

**Ports:** 3001 (development/production)

### 1.4.1 API Endpoints

Base path: `/api`

**Users**

| | |
|---|---|
| GET | /users |
| GET | /users/:id |
| GET | /users/search |
| POST | /users |
| PUT | /users/:id |
| DELETE | /users/:id |

**Projects**

| | |
|---|---|
| GET | /projects |
| GET | /projects/:id |
| POST | /projects |
| PUT | /projects/:id |
| DELETE | /projects/:id |

**Response Behavior**

| Status Code | Condition |
|---|---|
| 200 | Successful retrieval or update |
| 201 | Successful creation |
| 404 | Resource not found |
| 500 | Server or database error |

### 1.4.2 Implementation

Three-layer architecture: Routes → Controllers → Models.

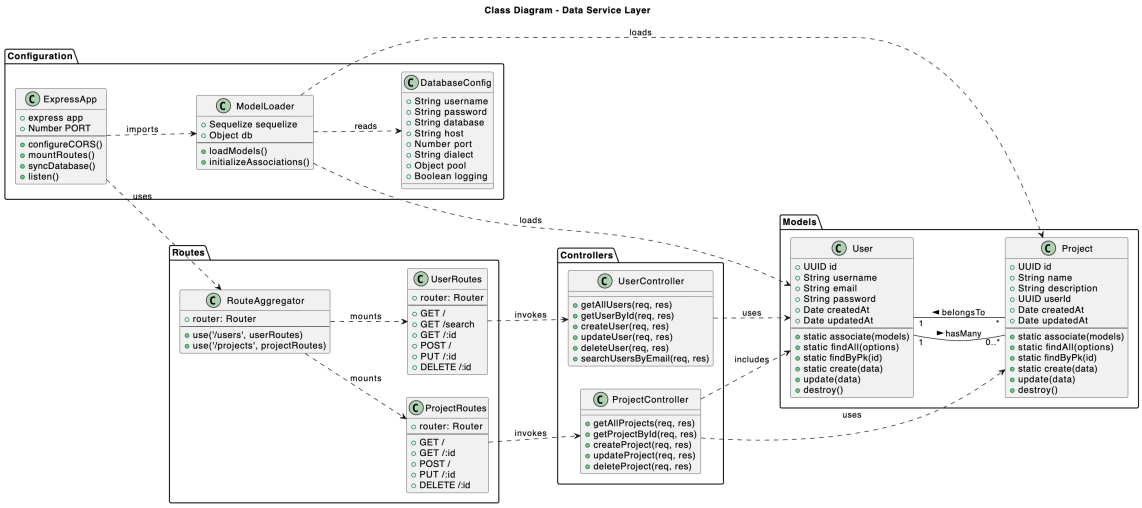



Figure 3: Class Diagram: Data Service Layer showing Routes, Controllers, and Models with Sequelize associations.

**Core Classes**

| Class | Module | Responsibility |
|---|---|---|

| | | |
|---|---|---|
| ExpressApp | `index.js` | Express server. Configures CORS, mounts routes, syncs database. |
| ModelLoader | `models/index` | Dynamic model loading. Initializes Sequelize, establishes associations. |
| User | `models/user` | User schema. UUID PK, unique email/username, hasMany Projects. |
| Project | `models/project` | Project schema. UUID PK, belongsTo User with cascade delete. |
| UserController | `controllers` | User CRUD handlers. Wraps Sequelize operations, excludes password. |
| ProjectController | `controllers` | Project CRUD handlers. Includes related User on retrieval. |
| RouteAggregator | `routes/index` | Mounts UserRoutes at `/users`, ProjectRoutes at `/projects`. |
| DatabaseConfig | `config/database` | Connection pool config. Reads environment variables. |

**Request Flow**

HTTP Request → CORS Middleware → RouteAggregator → Route → Controller → Sequelize Model → PostgreSQL → JSON Response

# 2 Validation and Verification

## 2.1 Backend API (arden-studio-api)

### 2.1.1 Unit Tests

Python test suite with 236 passing tests achieving 77% overall coverage. Tests validate agents, routes, tools, prompts, and utility modules.

### 2.1.2 Smoke Tests

**Status:** Planned

Route integration is not yet complete. Once finalized, a Postman collection will be

```
============================= tests coverage ==============================
_____________ coverage: platform darwin, python 3.12.3-final-0 _____________

Name                                                 Stmts   Miss Branch BrPart  Cover
------------------------------------------------------------------------------------
src/agents/__init__.py                                   0      0      0      0   100%
src/agents/agent.py                                     19      0      0      0   100%
src/agents/bases/__init__.py                             2      0      0      0   100%
src/agents/bases/precontex_agent_base.py               207     46     86      8    79%
src/agents/lore_bible_agent/__init__.py                  0      0      0      0   100%
src/agents/lore_bible_agent/agent.py                    15      2      0      0    87%
src/agents/narrative_graph_agent/__init__.py             2      0      0      0   100%
src/agents/narrative_graph_agent/agent.py               15      2      0      0    87%
src/config/__init__.py                                   2      0      0      0   100%
src/config/chat_model.py                                19      0      2      0   100%
src/core/__init__.py                                     0      0      0      0   100%
src/core/app.py                                          6      0      0      0   100%
src/core/app_factory.py                                 21      0      4      2    92%
src/prompts/__init__.py                                  5      0      0      0   100%
src/prompts/base.py                                     14      1      2      0    94%
src/prompts/lore_bible_prompt.py                         6      0      0      0   100%
src/prompts/narrative_graph_prompt.py                    6      0      0      0   100%
src/prompts/supervisor_prompt.py                         9      1      0      0    89%
src/routes/__init__.py                                   0      0      0      0   100%
src/routes/chat.py                                      85     60     14      1    26%
src/routes/session.py                                   40      0      6      0   100%
src/tools/__init__.py                                    9      1      0      0    89%
src/tools/base.py                                       37      1      8      0    98%
src/tools/edit_file.py                                  56      4     22      2    92%
src/tools/exceptions.py                                 12      0      0      0   100%
src/tools/langchain_tools.py                            33     19      6      0    36%
src/tools/read_file.py                                  37      6     14      0    84%
src/tools/write_file.py                                 25      8      6      1    65%
src/utils/__init__.py                                    5      0      0      0   100%
src/utils/get_openai_model.py                           16      5     10      5    62%
src/utils/load_file.py                                  23      6      6      2    72%
src/utils/prompt_utils.py                                7      0      0      0   100%
src/utils/save_file.py                                  33      9      8      3    71%
------------------------------------------------------------------------------------
TOTAL                                                  766    171    194     24    77%
========================== 236 passed in 2.37s ===========================
$
```

Figure 4: Backend API unit test coverage report.

created to validate all API endpoints. The smoke test will verify:

- All endpoints return expected status codes
- Request/response schemas match specifications
- Authentication and session management function correctly
- Proxy routes to Data Service operate as expected

### 2.1.3 Agent Performance and Metrics

**Status:** Planned

Evaluation framework for multi-agent system responses. Test methodology:

1. Prepare a set of representative user queries with expected agent actions
2. Execute queries against the supervisor agent
3. Evaluate responses based on:

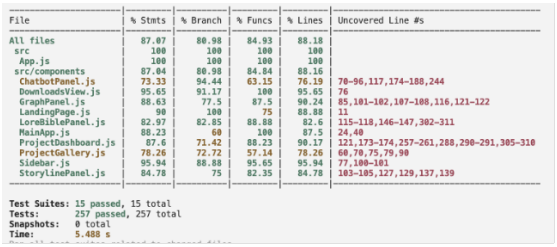
    - **Output Generation**: Whether a valid response was produced
    - **Flow Completion**: Whether all expected agent handoffs and tool calls occurred
    - **Hallucination Detection**: Whether responses contain fabricated or inconsistent information

4. Rank agent performance according to accuracy and reliability metrics

## 2.2 Frontend (arden-studio-web)

**Unit Tests**
257 passing tests. Coverage: 87% statements, 81% branches, 85% functions, 88% lines.

**Performance Tests**
Chrome DevTools, cache disabled. DOMContentLoaded: 196ms, full load: 250ms, 34 requests.



| File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|---|---|---|---|---|---|
| All files | 87.07 | 80.98 | 84.93 | 88.18 | |
| src | 100 | 100 | 100 | 100 | |
| App.js | 100 | 100 | 100 | 100 | |
| src/components | 87.04 | 80.98 | 84.84 | 88.16 | |
| ChatbotPanel.js | 73.33 | 94.44 | 63.15 | 76.19 | 70-96,117,174-188,244 |
| DownloadsView.js | 95.65 | 91.17 | 100 | 95.65 | 76 |
| GraphPanel.js | 88.63 | 77.5 | 87.5 | 90.24 | 85,101-102,107-108,116,121-122 |
| LandingPage.js | 90 | 100 | 75 | 88.88 | 11 |
| LoreBiblePanel.js | 82.97 | 82.85 | 88.88 | 82.6 | 115-118,146-147,302-311 |
| MainApp.js | 88.23 | 60 | 100 | 87.5 | 24,40 |
| ProjectDashboard.js | 87.6 | 71.42 | 88.23 | 90.17 | 121,173-174,257-261,288,290-291,305-310 |
| ProjectGallery.js | 78.26 | 72.72 | 57.14 | 78.26 | 60,70,75,79,90 |
| Sidebar.js | 95.94 | 88.88 | 95.65 | 95.94 | 77,100-101 |
| StorylinePanel.js | 84.78 | 75 | 82.35 | 84.78 | 103-105,127,129,137,139 |

Test Suites: 15 passed, 15 total
Tests: 257 passed, 257 total
Snapshots: 0 total
Time: 5.488 s

Figure 5: Frontend unit test coverage.

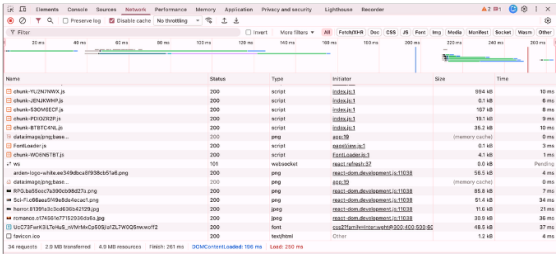
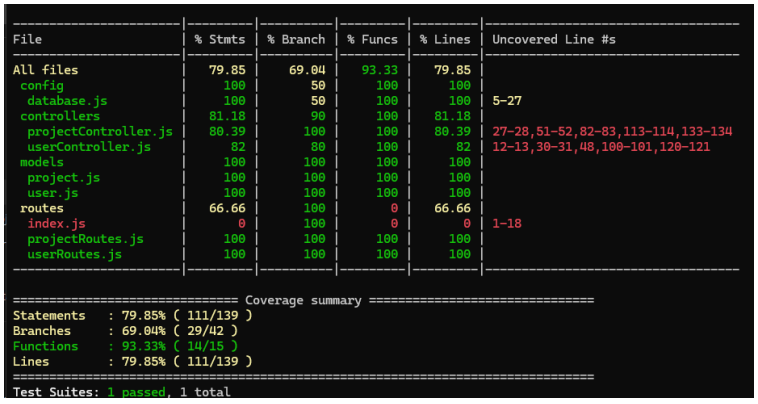


Figure 6: Frontend performance metrics.

## 2.3 Data Service (arden-data-service)

### 2.3.1 Unit Tests

Jest-based test suite with 33 passing tests achieving 79.85% coverage. Models at 100%, controllers at 81%, routes at 100%.



| File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|---|---|---|---|---|---|
| All files | 79.85 | 69.04 | 93.33 | 79.85 | |
| config | 100 | 50 | 100 | 100 | |
| database.js | 100 | 50 | 100 | 100 | 5-27 |
| controllers | 81.18 | 90 | 100 | 81.18 | |
| projectController.js | 80.39 | 100 | 100 | 80.39 | 27-28,51-52,82-83,113-114,133-134 |
| userController.js | 82 | 80 | 100 | 82 | 12-13,30-31,48,100-101,120-121 |
| models | 100 | 100 | 100 | 100 | |
| project.js | 100 | 100 | 100 | 100 | |
| user.js | 100 | 100 | 100 | 100 | |
| routes | 66.66 | 100 | 0 | 66.66 | |
| index.js | 0 | 100 | 0 | 0 | 1-18 |
| projectRoutes.js | 100 | 100 | 100 | 100 | |
| userRoutes.js | 100 | 100 | 100 | 100 | |

Coverage summary
Statements : 79.85% ( 111/139 )
Branches : 69.04% ( 29/42 )
Functions : 93.33% ( 14/15 )
Lines : 79.85% ( 111/139 )
Test Suites: 1 passed, 1 total

Figure 7: Data Service test coverage report.

# References

1. Brown, S. (2024). The C4 Model for Visualising Software Architecture. *System Context Diagram*. Available at: C4 Model: System Context Diagram
2. PlantUML Standard Library. (2024). C4-PlantUML: C4 Model Diagrams with PlantUML. GitHub Repository. Available at: GitHub: C4-PlantUML
3. SequenceDiagram.org. (2024). Online Sequence Diagram Tool. Available at: SequenceDiagram.org

# Appendices

## UI Screenshots

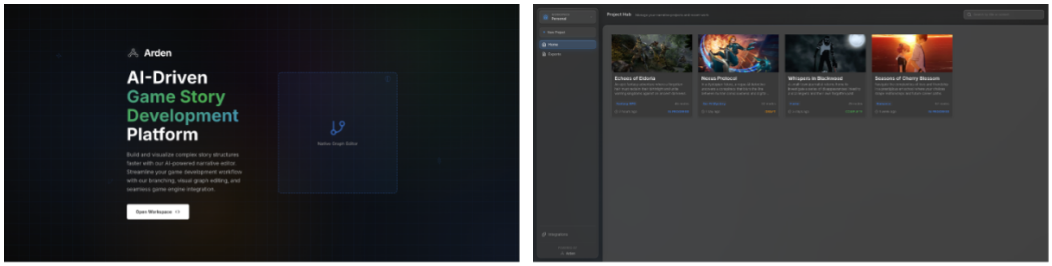



Figure 8: Landing Page (left) and Project Hub (right)

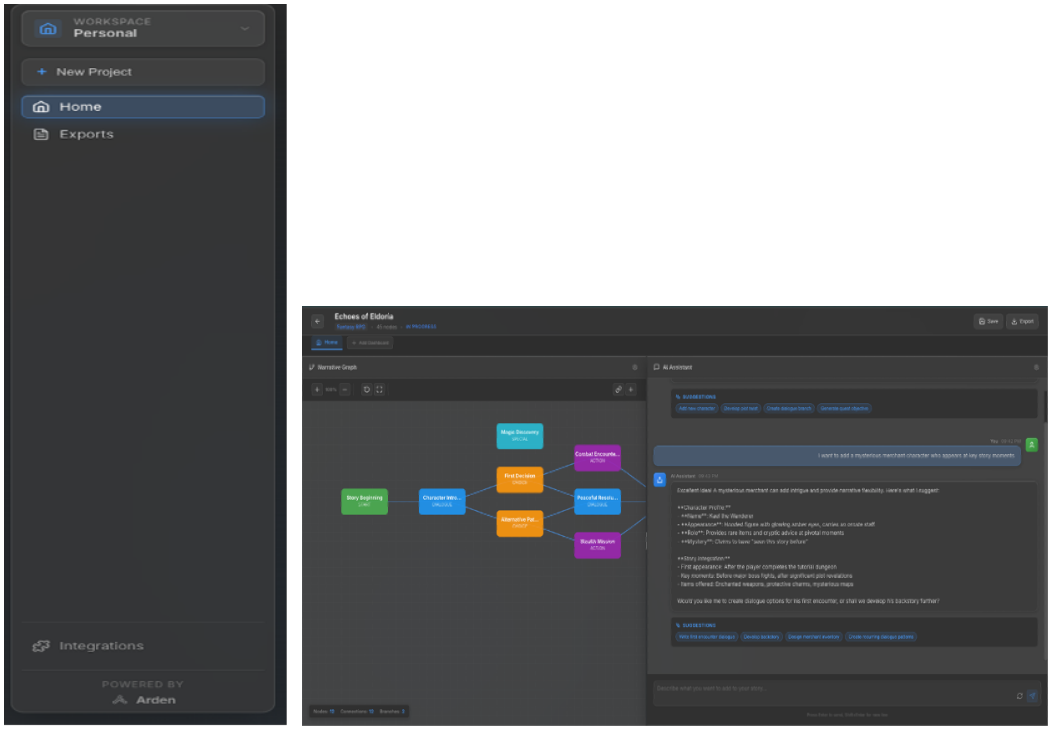



Figure 9: Workspace Sidebar (left) and Project Dashboard (right)

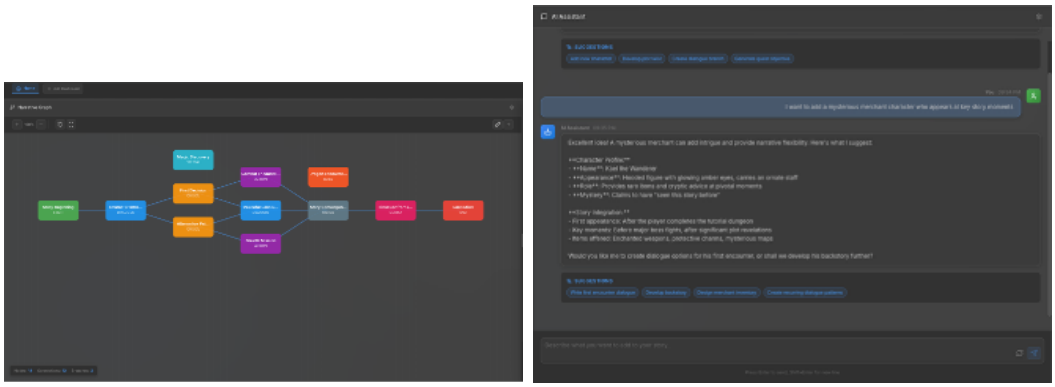


Figure 10: Narrative Graph (left) and AI Assistant (right)

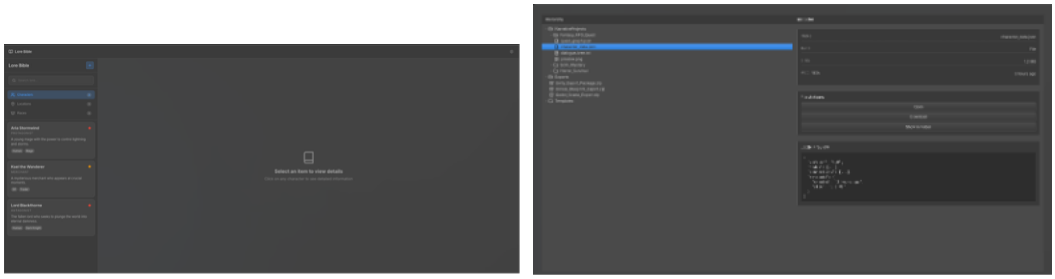


Figure 11: Lore Bible (left) and Storyline Panel (right)

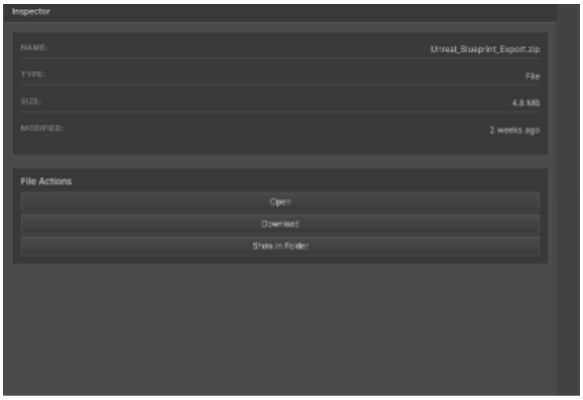



Figure 12: Exports View

## Sequence Diagrams

The following sequence diagrams illustrate the key interaction flows within the ARDEN Studio system.

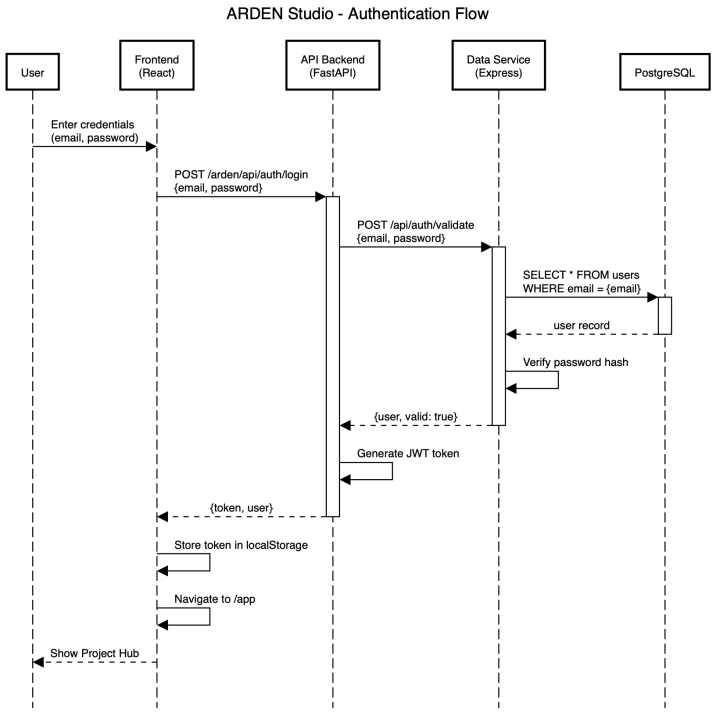



Figure 13: Authentication Flow: User login process through frontend, API backend, data service, and database.

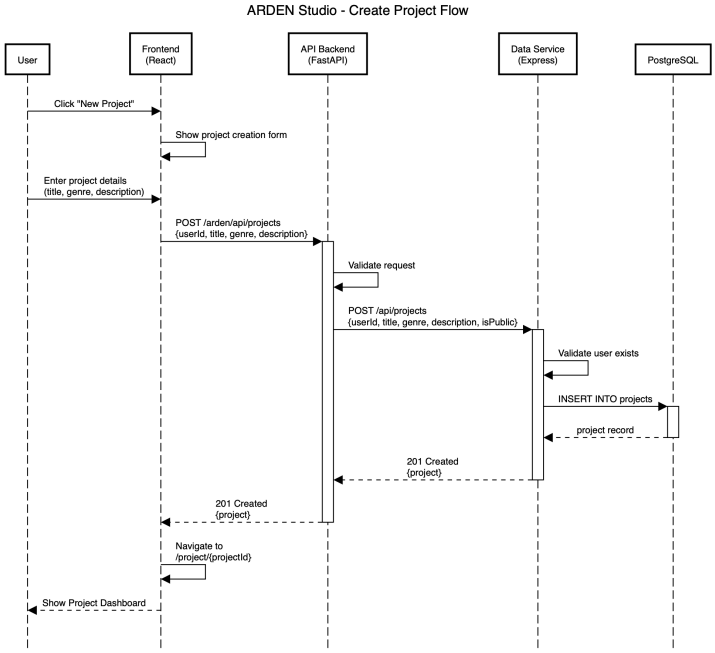



Figure 14: Create Project Flow: New project creation from form submission to database persistence.

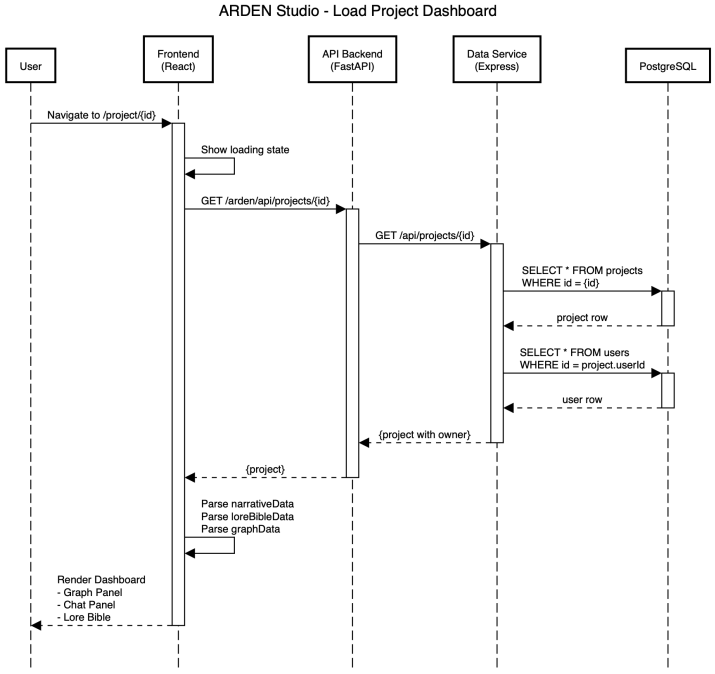



Figure 15: Load Project Dashboard: Project data retrieval and dashboard rendering process.

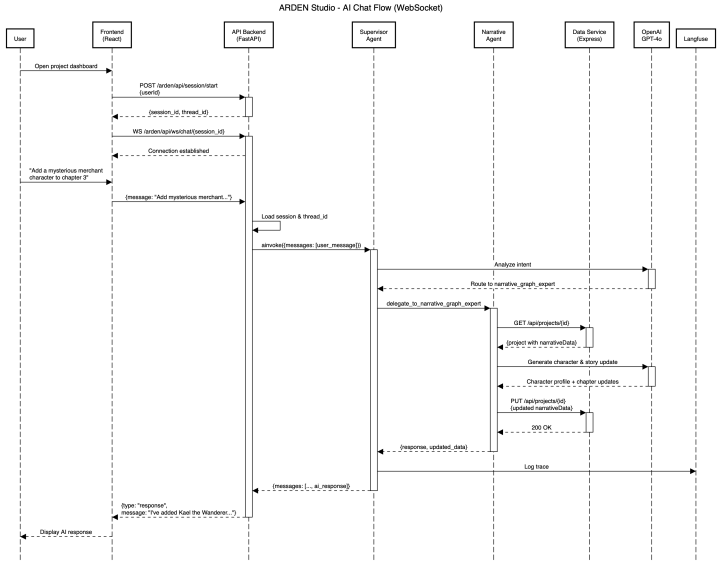



Figure 16: AI Chat Flow (WebSocket): Real-time chat interaction with multi-agent orchestration.

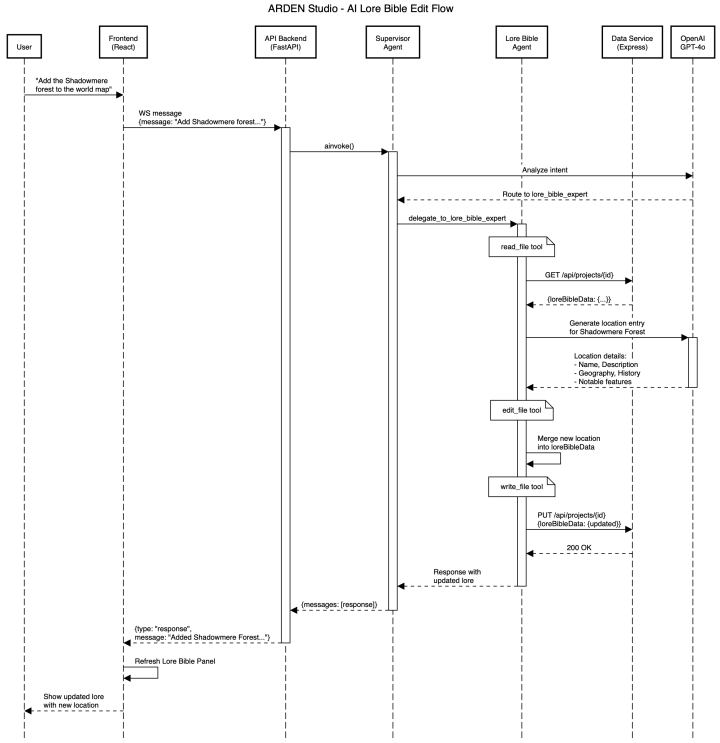



Figure 17: AI Lore Bible Edit Flow: Lore bible modification through AI agent with tool execution.

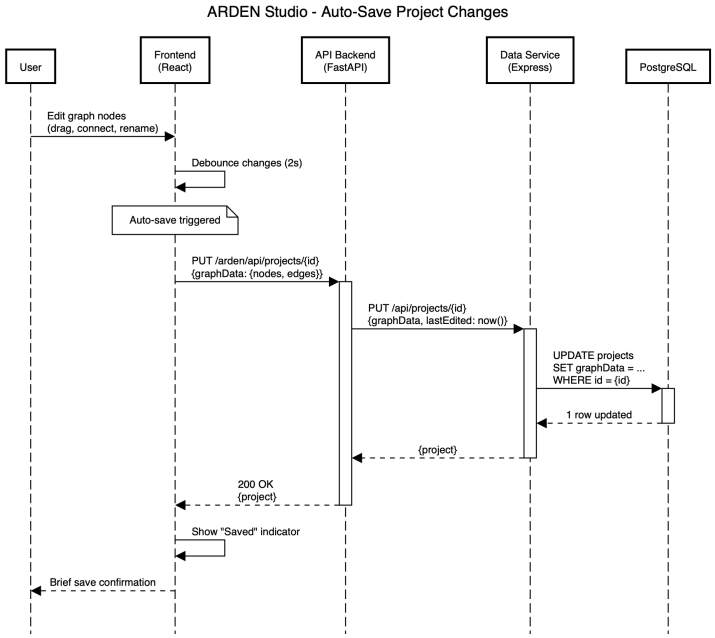



Figure 18: Auto-Save Project Changes: Debounced auto-save mechanism for graph edits.